🔥

ReSurge Product Requirements Document

Version 1.0 · March 2026 · Internal

1. Product Overview

Product Name: ReSurge

Product Type: Reddit Monitoring & Engagement Platform

Description: ReSurge is an automated Reddit post monitoring platform that detects viral posts in real time and instantly notifies account holders — so they never need to manually search Reddit. Holders receive a direct email alert with an "Open & Respond" link that auto-logs them in and surfaces an AI-generated comment suggestion. The full team — holders, monitors, and the main admin — operates from a single unified web portal.

---

2. Problem Statement

Teams managing Reddit accounts face two core problems:

- **Time wasted searching:** Staff manually browse Reddit to find posts worth commenting on. This is slow, unreliable, and inconsistent — good posts are missed or caught too late.
- **Inconsistent engagement quality:** Even when posts are found, deciding what to write and when to post varies widely across the team. Without structured guidance and tracking, comment quality and timing degrade.

ReSurge solves both: it finds the right posts automatically, delivers an AI comment suggestion directly to the holder, and tracks every action from alert to posted comment in one place.

---

3. Target Users

- **Holders (NPLs, CPLs, Interns)** — Staff who manage Reddit accounts. They receive viral post alerts via email and use the holder view to review AI-generated comment suggestions and record posted comments.
- **Monitors (CPLs)** — Senior team members assigned to oversee specific holders. They track notification history, comment conversions, and account activity across their assigned holders without needing admin access.
- **Main Admin** — Office-side operator with full system control. Manages subreddits, detection thresholds, users, and monitors the entire team from one dashboard.

---

4. Product Goals

1. Eliminate manual Reddit searching — notify holders within 60 seconds of a post reaching viral momentum
2. Reduce comment quality variance by delivering AI-generated suggestions directly in the alert flow
3. Give monitors full visibility into their assigned holders' activity without needing admin access
4. Allow admins to tune detection sensitivity per subreddit and manage the full team from one place
5. Track holder performance over time via a leaderboard (posts, upvotes, activity streaks)

---

5. Core Functional Requirements

5.1 Stack Engine (S1 → S3 Detection Pipeline)

Goal: Automatically identify posts gaining viral momentum before they peak.

- **Three-Stage Pipeline:** Posts move through S1 (Discovery) → S2 (Evaluation) → S3 (Viral Alert). Posts that fail criteria at any stage are discarded silently.
- **S1 — Discovery:** All new posts from tracked subreddits are fetched every 60 seconds. Posts failing minimum engagement (default: 20) after a minimum age (default: 10 min) are discarded.
- **S2 — Evaluation Window:** Posts passing S1 enter a configurable growth window (default: 7–14 minutes). An engagement snapshot is taken at window open. At window close, posts not meeting the growth threshold (default: 30%) are discarded.
- **S3 — Viral Alert:** Posts passing S2 trigger immediate email notifications to all subscribed holders. S3 posts auto-expire after 4 hours.
- **Poll Cycle:** A single global BullMQ job runs every 60 seconds, covers all active subreddits, refreshes S1/S2 engagement in one batched Reddit API call, and runs all stack transitions in sequence.
- **Configurable Thresholds:** All five parameters (`s1MinAge`, `s1MinEng`, `s2EvalStart`, `s2EvalEnd`, `s2GrowthPct`) are adjustable per subreddit via threshold sliders in the admin dashboard. A global default applies to subreddits without per-subreddit overrides. Changes take effect on the next poll cycle.
- **Threshold Audit Log:** Every slider change is recorded in `threshold_edits` with before/after JSON snapshots, visible in the All Edits view.

5.2 Subreddit Management

Goal: Admin controls which subreddits are monitored.

- Add a subreddit → polling starts on the next cycle. Admin sees a confirmation toast.
- Remove a subreddit → polling stops; all historical post and notification data is preserved (soft-delete). Requires confirmation modal.
- Pause a subreddit → polling temporarily stops without removing it. Resume restarts from the next cycle.
- Hide from holders → subreddit remains tracked but is not visible in the holder portal for subscriptions.

5.3 Notification & Alert System

Goal: Deliver actionable alerts to holders the moment a post reaches S3.

- **Email Delivery:** Sent via Resend from a verified sender domain. Each email contains the post title, subreddit, and an "Open & Respond" CTA button.

- **Deep-Link Auto-Login:** The CTA contains a pre-signed JWT. Clicking it opens the portal, logs the holder in automatically, and surfaces the post modal — no manual login required.
- **Deduplication:** One notification per user per S3 post, regardless of how many holder accounts they have subscribed to that subreddit.
- **Status Lifecycle:** `sent → opened → posted`. When a holder submits their posted comment URL, `postedLink` and `postedAt` are recorded.
- **Notification Pause:** Holders can pause notifications for a set duration. The pollWorker skips paused holders and logs the remaining pause time.
- **Failed Delivery Tracking:** If Resend rejects the email, the notification status is set to `failed:{error_message}` for diagnosis in the All Notifications view.

5.4 AI Comment Generation

Goal: Reduce the effort and judgment required to write a relevant comment.

- Triggered with one click from the S3 feed (admin) or any notification (holder/monitor).
- The system first fetches the top 8 existing Reddit comments to calibrate tone and context.
- OpenAI GPT-4o generates multiple natural, human-sounding suggestions — prompted to avoid AI-sounding openers and clichés.
- Suggestions are cached per post (`generated_comments` table) to avoid redundant API calls on repeated opens.

5.5 Unified Portal — Role-Based Views

Goal: A single web application that adapts its interface based on the logged-in user's role. All three roles access the same URL; the portal renders the correct view after authentication.

Main Admin View

- **Subreddit Tracker:** Real-time S1–S3 post feed per subreddit. Stack badges show live counts. S3 viral feed auto-refreshes every 15 seconds.
- **Threshold Sliders:** Per-subreddit sliders for all 5 detection parameters, saved immediately.
- **All Notifications:** Full notification history across all users, subreddits, and statuses.
- **All Edits:** Audit log of all threshold changes with before/after values and timestamps.
- **Manage Subreddits:** Add, remove, pause, and toggle holder visibility per subreddit.

- **Monitors:** Create monitor accounts and assign them to specific holders.
- **Holders:** View all holder accounts with their Reddit/email pairs and subreddit subscriptions.
- **Users:** Invite-based user management — admin adds email + role to the invite list; user signs in with Google and is automatically matched. Pending invites shown with a badge count. Admin can deactivate or soft-delete users.
- **Leaderboard:** Ranked performance table for all holders and monitors — total posts, upvotes earned, last-24h activity, avg posts/day, upvote rate, active days. Rebuilt daily at 09:30 UTC.

Monitor View

- Overview cards for each assigned holder with quick stats: notifications received, comments posted, active accounts.
- Holder detail drill-down: account pills, time-range filterable stats, subreddit list, full notification history with status filters and clickable post/comment links.
- Leaderboard view (read-only).
- Monitors only see holders explicitly assigned to them — not the full team.

Holder View

- Notification feed sorted by recency with status badges. Filterable by status.
- Post detail modal: title, subreddit, engagement stats, AI comment suggestions (generated on demand), and a "Mark as Posted" form to record the comment URL.
- Account management: manage email/Reddit pairs and subscribe each account to subreddits.
- Leaderboard view (read-only).
- Notification pause control: pause alerts for a chosen duration directly from the portal.

5.6 Leaderboard & Performance Tracking

Goal: Give admins and monitors visibility into holder output and quality over time.

- Per-holder metrics: total comments posted, total upvotes earned, comments in last 24 hours, avg posts/day since first post, upvote rate (upvotes per post), number of active days.
- Comment upvote scores are fetched from Reddit's API and stored in `comment_scores`.
- Leaderboard cache is rebuilt daily at 09:30 UTC (3 PM IST) by the pollWorker.

- Visible to all three roles; admins and monitors see the full team ranking.

---

6. Authentication & Access Control

- **Google OAuth only:** All users sign in with their Google account via Google Identity Services (GSI). No email/password login.
- **Invite-based access:** Only emails pre-added to the `invited_users` table by an admin can sign in. Uninvited Google accounts are rejected at login.
- **Role assignment:** Admin sets the role (main, monitor, holder) when inviting a user. On first login, the account is created with that role automatically.
- **Multi-role support:** A user can hold multiple roles simultaneously (stored as a `roles` array). On login, they select which role view to use.
- **JWT sessions:** After Google OAuth, the backend issues a 30-day JWT. Deep-link notification emails contain short-lived role-scoped JWTs for auto-login.
- **Deactivation:** Admin can deactivate or soft-delete users. Deactivated/deleted users are rejected at login.

---

7. Data Model

subreddits

- `id, name, isActive, isPaused, visibleToHolders, addedAt, subscribers`

posts

- `id, redditId, subreddit, title, url, author, selftext, upvotes, comments, engagement, redditCreatedAt, stackEnteredAt, stack (1–3), engAtStackEntry, engAtEvalStart, lastGrowth, alertedAt, discarded, createdAt, updatedAt`

thresholds

- `id, subreddit (null = global default), s1MinAge, s1MinEng, s2EvalStart, s2EvalEnd, s2GrowthPct, updatedAt`

threshold_edits

- `id, subreddit, before (JSON snapshot), after (JSON snapshot), note, editedAt`

users

- `id, email, passwordHash, name, role, roles (text[]), phone, isActive, isDeleted, adminAcknowledged, notificationsPausedUntil, createdAt`

invited_users

- `id, email, role, invitedAt`

holder_accounts

- `id, holderId, emailAddress, redditUsername, notes, subreddits (text[]), addedAt`

monitor_assignments

- `id, monitorId, holderId, assignedAt`

notifications

- `id, userId, postId, accountId, subreddit, postTitle, postUrl, sentAt, openedAt, status (sent | opened | posted | failed:msg), postedLink, postedAt`

generated_comments

- `id, postId, provider (openai), suggestions (JSON), createdAt`

comment_scores

- `id, notificationId, score, fetchedAt`

leaderboard_cache

- `id, userId, name, role, totalPosted, totalUpvotes, last24hPosted, avgPerDay, upvoteRate, activeDays, firstPostedAt, updatedAt`

user_subreddits

- `id, userId, subreddit, joinedAt`

---

8. Tech Stack

Backend

- **Runtime:** Node.js 20+ · TypeScript (ESM)
- **Framework:** Hono 4.x

- **Database:** PostgreSQL with Drizzle ORM (migrations via drizzle-kit)
- **Job Queue:** BullMQ + Redis — one global repeatable poll job every 60 seconds
- **Auth:** Google OAuth via `google-auth-library` · JWT (30-day) via `jsonwebtoken`
- **Email:** Resend SDK (v6.x) — sent from verified sender domain
- **AI:** OpenAI GPT-4o for comment generation
- **Deployment:** Railway — auto-deploys from GitHub `main` branch via Nixpacks

Frontend

- **Single unified portal** serving all three roles (Main Admin, Monitor, Holder)
- **Framework:** React 18 + Vite + TypeScript
- **Auth:** Google Identity Services (GSI) — "Sign in with Google" button
- **Deployment:** Vercel — auto-deploys from GitHub `main` branch

Infrastructure

- PostgreSQL — Railway managed database
- Redis — Railway managed (required by BullMQ for job scheduling)
- Environment variables managed in Railway dashboard (backend) and Vercel dashboard (frontend)

---

9. Key Metrics Tracked

- **Stack Distribution:** Live count of posts in S1, S2, S3 per subreddit
- **Alert Rate:** S3 alerts triggered per subreddit per day
- **Conversion Rate:** Notifications sent vs. comments posted, per holder and per subreddit
- **Response Time:** Time between `sentAt` and `openedAt` / `postedAt` per notification
- **Holder Performance:** Total posts, upvotes earned, avg posts/day, active days — via daily leaderboard rebuild

---

10. Success Metrics

1. Holder receives an alert within 60 seconds of a post reaching S3

2. Holder can open an alert, review the AI comment, and mark it as posted in under 2 minutes
3. Admin can add a subreddit and see live S1–S3 data within 2 minutes
4. Monitor can view a holder's full notification history without needing admin access
5. Threshold changes on a subreddit take effect within one poll cycle (60 seconds)
6. Leaderboard reflects the previous day's activity by 3 PM IST daily

---

11. Assumptions & Dependencies

Assumptions

- Target subreddits are publicly accessible (no private or restricted communities)
- Post engagement (upvotes + comments) is a reliable early signal of viral potential
- All team members have Google accounts matching their invited email addresses
- Subreddits tracked rarely exceed 25 new posts per 60-second poll cycle

External Dependencies

- **Reddit public JSON API** — unauthenticated. Railway's datacenter IP may receive 403s from Reddit's bot protection on certain endpoints.
- **Resend** — verified sender domain required. Sending from `onboarding@resend.dev` is restricted to the Resend account owner's email only and will fail for all other recipients.
- **OpenAI GPT-4o** — API key with GPT-4o access required for comment generation.
- **Google OAuth** — Google Client ID configured in both Railway (backend) and Vercel (frontend) environment variables.
- **Redis** — BullMQ requires a live Redis connection. If Redis is unavailable, the poll worker will not run and no alerts will be sent.
- **Reddit spam/review hold** — New posts may be delayed 15–30 minutes before appearing in Reddit's API. This is a Reddit platform constraint outside the system's control.